Smart Campus Assistant Software Requirements Specification (SRS)
Version 1.0 | March 2026

by Noam Freiman

1. System Purpose

1.1 Vision Statement Smart Campus Assistant is an AI-powered web platform that allows students to manage their academic lives in one place: asking questions, building a personal schedule, tracking grades, and calculating target scores, through a chat interface powered by an AI model.

1.2 The Problem Students waste time searching for information scattered across multiple platforms: exam dates on one site, office hours in an email, class schedules in a separate system. There is no single tool that brings all this information together and allows natural language questions with instant answers.

1.3 The Solution A web system with an AI chatbot connected to a campus database. Students get instant answers, an automatically built personal weekly schedule with conflict detection, and target grade calculations based on existing scores. Teachers manage all information through a dedicated interface.

1.4 System Scope

The system does:

- Answers campus questions in natural language
- Builds a personal weekly schedule and detects conflicts
- Calculates target grade based on existing scores
- Exports personal schedule as a PDF

The system does not:

- Replace the official course registration system
- Send automatic notifications or alerts
- Handle payments, scholarships, or course enrollment

1.5 Stakeholders

Student Primary User. Provides questions, personal events, and target grades. Receives instant answers, a personal schedule, and grade calculations.

Lecturer Data Manager. Provides class schedules, exam details, and office hours. Receives fewer repeated questions from students.

University Admin System Client. Provides institutional approval and data access. Receives better student experience and system performance

1.6 System Data Management

Admin Setup The system administrator creates accounts for all students and teachers. The admin also connects students to their specific courses in the database.

Teacher Data Entry At the start of each week or month, the teacher updates the course information. The teacher adds class schedules, exam dates, and office hours to the database so the AI has the right information.

Student Usage The student logs in to ask the AI questions based on the data the teachers entered. The student only types in personal details, like a target grade or a private event, to calculate scores or build a personal schedule.

1.7 Database System Tables

Students store student login credentials, password, and academic profile. (PK: student_id)
Teachers store teacher login credentials, password, and department information. (PK: lecturer_id).
Courses stores course details and links each course to its teacher. (PK: course_id, FK: lecturer_id).
Exams store exam dates, times, and locations per course. (PK: exam_id, FK: course_id)
Lessons store recurring class schedules per course. (PK: lesson_id, FK: course_id).
Office Hours stores each teacher's available meeting times. (PK: office_hour_id, FK: lecturer_id).
Grades store student grades per course and exam type. (PK: grade_id, FK: student_id, FK: exam_id).
Student Courses link each student to their enrolled courses. (PK: registration_id, FK: student_id, FK: course_id).

2. Use Cases

UC-01 | Exam Date Question

- **Actor:** Student
- **Pre-condition:** Student is logged in, The lecturer has assigned a room location in the system
- **Description:** Student asks when an exam is for a specific subject. System queries the Exams table and returns the date, time, and location.
- **Post-condition:** System displays a structured response with date, time, and room.
- **Edge Case:** Exam not found in database. The system outputs a standard missing data message ("Exam date not yet updated") and retrieves the lecturer's email from the Teachers table for contact.

UC-02 | Room Location Question

- **Actor:** Student
- **Pre-condition:** Student is logged in. The lecturer has assigned a room location in the system
- **Description:** Student asks where a class or exam is held. System queries the Lessons or Exams table.
- **Post-condition:** System displays building, floor, and room.
- **Edge Case:** Location field is null. The system returns a default response directing the student to contact the department secretary.

UC-03 | Office Hours Question

- **Actor:** Student
- **Pre-condition:** Student is logged in. The lecturer has set office hours in the system.
- **Description:** Student queries a specific lecturer's office hours. The system pulls data from the Office Hours table.
- **Post-condition:** System displays available days and times.
- **Edge Case:** No office hours defined for the requested lecturer. The system retrieves and displays the lecturer's email address for direct contact.

UC-04 | Class Schedule for Specific Course

- **Actor:** Student
- **Pre-condition:** Student is logged in. The course exists in the system and the lecturer has updated the class schedule.
- **Description:** Student requests the schedule for a specific course. The system retrieves all recurring class dates from the Lessons table.
- **Post-condition:** System outputs an organized list of class dates and times.
- **Edge Case:** Ambiguous or partial course name entered. System queries the Courses table for partial matches and displays a list of clickable options for the user to select.

UC-05 | Target Grade Calculator

- **Actor:** Student
- **Pre-condition:** Student is logged in. Partial grades exist in the DB.
- **Description:** Student enters a target grade. The system retrieves current grades from the Grades table and calculates the mathematical minimum score needed in remaining exams. If no grades exist, the UI allows inputting hypothetical grades for a local sandbox calculation (data is not written to the DB).
- **Post-condition:** System displays the target grade, current average, and required score.
- **Edge Case:** Target grade is mathematically impossible based on current average. The system displays an error indicating the maximum possible grade that can still be achieved.

UC-06 | Personal Schedule Builder

- **Actor:** Student
- **Pre-condition:** Student is logged in.
- **Description:** Student triggers the schedule builder. The system pulls enrolled course schedules from the Lessons table and merges them with temporary, session-based personal events input by the user.
- **Post-condition:** UI renders a weekly table. Clicking "Save to My Schedule" generates a downloadable PDF.
- **Edge Cases:**
- **1.** Time slot overlap between a class and a personal event: The UI highlights the conflicting time slots in red to alert the user. The system does not automatically resolve the conflict.
- **2.** Lecturer has not entered schedule data: System displays a warning that the specific course data is missing and excludes it from the rendered schedule.

UC-07 | Export Schedule to PDF

- **Actor:** Student
- **Pre-condition:** The schedule is saved on the schedule page after the student created it.
- **Description:** The student clicks the download to PDF button.
- **Post-condition:** A PDF file of the schedule is generated and downloaded.

-

3. Non-Functional Requirements (NFR)

- **NFR-01 | Performance (Response Time):** The system shall display a visual loading indicator (Spinner) within 1 second of query submission. The final AI-generated response must be delivered to the user within a maximum of 30 seconds.
- **NFR-02 | Language and Accessibility** The application shall feature a full English interface, including all database content and AI responses. The interface will maintain a minimum font size of 16px.
- **NFR-03 | Security:** Access is restricted to registered users within the Students table. The system shall implement an automatic session timeout after 24 hours of inactivity.
-
- **NFR-04 | Efficiency (User Experience):** Generating a personal schedule and exporting it to a pdf file shall require a maximum of 3 clicks from the moment the student enters the schedule builder screen.

4. System Process Flow

Step 1: Submit question: The student types a question in the application and sends it.

Step 2: Show loading indicator: The application shows a loading spinner within one second so the student knows the system is working.

Step 3: Send question request: The application sends the text question and the student ID to the backend server.

Step 4: Classify: The server checks who the student is and decides what type of question it is, like a schedule, grades, or general question.

Step 5: Query relevant data: The server asks the database for the specific information needed for that student and that question category.

Step 6: Return data records: The database finds the relevant information and sends it back to the server.

Step 7: Send prompt: The server packages the student question, the database records, and the system rules together, and sends them to the AI service.

Step 8: Return generated answer: The AI reads the information and sends a generated answer back to the server.

Step 9: Validate answer quality: The server checks the AI response. It looks for a specific fallback keyword to make sure the AI actually found an answer in the data and did not guess

Step 10: Return structured response: The server sends the final, safe answer and its category back to the application layer.

Step 11: Render answer to user : The application displays the final answer to the student.

4.1 Requirements Conflict Performance Speed vs AI Quality, On one hand, requirement NFR-01 states that the system must return a final answer within 30 seconds at most. On the other hand, the system needs to run the AI model and perform quality validation using the fallback mechanism. The tension here is that if we require the AI to perform a very deep and accurate analysis, it might take more than 30 seconds, which would fail the performance requirement.

Smart Campus Assistant – UML Use Case Diagram

«system» **Smart Campus Assistant**

STUDENT	Student Use Cases	Lecturer & Manager Use Cases	ACTORS
☐ **Student**	UC-01 Ask Exam Date	UC-L1 Manage Class Schedule	☐ **Lecturer**
	UC-02 Ask Room Location	UC-L2 Update Exam Details	
	UC-03 Ask Office Hours	UC-L3 Set Office Hours	
	UC-04 View Course Schedule	UC-L4 Enter Student Grades	
	UC-05 Calculate Target Grade	UC-M1 Approve Lecturer Permissions	☐ **Campus Manager**
	UC-06 Build Weekly Schedule	UC-M2 Manage Campus Data	
	UC-07 Export Schedule to PDF	UC-M3 View System Reports	

Key Relationships

«include»: UC-06 (Build Weekly Schedule) ————► UC-07 (Export Schedule to PDF)

«extend»: UC-M1 (Approve Lecturer Permissions) - - - - ► UC-M2 (Manage Campus Data)

Student ——► **UC-01, UC-02, UC-03, UC-04, UC-05, UC-06, UC-07**

Lecturer ——► **UC-L1, UC-L2, UC-L3, UC-L4**

Campus Manager ——► **UC-M1, UC-M2, UC-M3**

⚠ Edge Cases

- UC-01: Exam not found → return lecturer email for manual query
- UC-02: Room null / not assigned → contact department office
- UC-03: No office hours set → return lecturer email + last-known slot
- UC-05: Target grade impossible → show max achievable grade
- UC-06: Time conflict detected → highlight red, no auto-resolution
- UC-M1: Approval pending → restrict Lecturer write access (UC-L1, L2, L3)

Legend

————► **Association**
Direct relationship between actor and use case

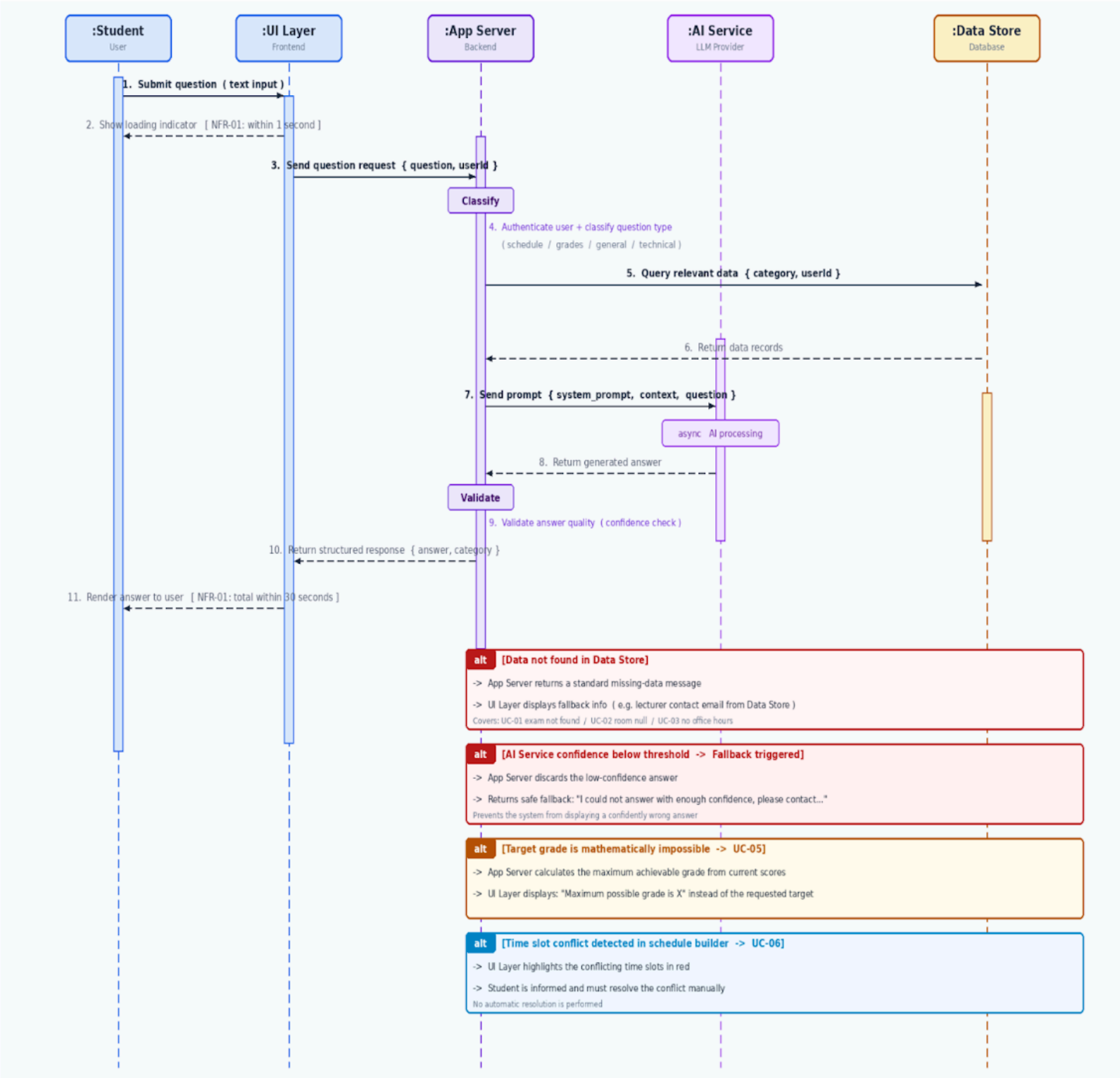
- - - - ► **«include»**
One use case always includes another

- - - - ► **«extend»**
One use case optionally extends another

(Oval) Use Case
A system function or feature

☐ **Actor**
An external entity that interacts with the system

Smart Campus Assistant

Diagram 2 of 2: Sequence Diagram

Student Submits Question AI Service Processes Response Returned